Experiment 2: Perceptron with Fixed Increment Learning Algorithm (Binary Step Function)

Question

To implement a perceptron using a fixed increment learning algorithm with the binary step activation function and train it until no further changes in weights are required.

Theory

The perceptron is an artificial neural network model used for binary classification. The binary step function is a simple activation function that outputs either 0 or 1 depending on whether the weighted sum is above a threshold. The perceptron updates its weights iteratively based on classification errors until convergence.

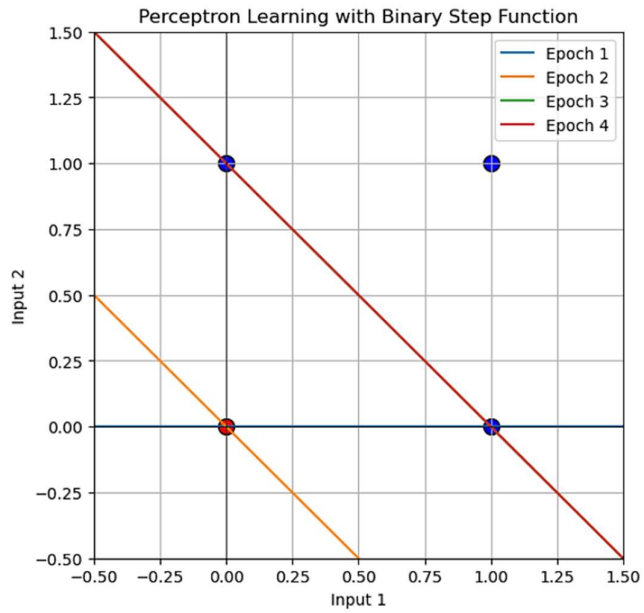
For an input vector $X = [x_1, x_2]$, weights $W = [w_1, w_2]$, and bias b, the perceptron computes: $y = f(WX + b)$ where f is the activation function defined as: $f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$

Implementation:

```
import numpy as np
import matplotlib.pyplot as plt

# Binary Step Activation Function
def binary_step(x):
    return 1 if x >= 0 else 0

# Training Data (OR Function)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 1])

# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
converged = False
iterations = 0
decision_boundaries = []

while not converged:
    iterations += 1
    previous_weights = weights.copy()
    previous_bias = bias

    for i in range(len(X)):
        weighted_sum = np.dot(X[i], weights) + bias
```

```
        prediction = binary_step(weighted_sum)
        error = y[i] - prediction
        weights += learning_rate * error * X[i]
        bias += learning_rate * error

    # Store decision boundary after each epoch
    if weights[1] != 0:
        x_vals = np.linspace(-0.5, 1.5, 100)
        y_vals = -(weights[0] * x_vals + bias) / weights[1]
        decision_boundaries.append((x_vals, y_vals))

    # Check for convergence
        if np.array_equal(previous_weights, weights) and previous_bias == bias:
            converged = True

# Plot decision boundary evolution
plt.figure(figsize=(6, 6))
for i, (x_vals, y_vals) in enumerate(decision_boundaries):
    plt.plot(x_vals, y_vals, label=f'Epoch {i+1}')

# Plot data points
for i in range(len(X)):
    plt.scatter(X[i][0], X[i][1], c='red' if y[i] == 0 else 'blue', marker='o', edgecolors='black', s=100)

plt.xlabel('Input 1')
plt.ylabel('Input 2')
plt.title('Perceptron Learning with Binary Step Function')
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.grid(True)
plt.show()

# Display final weights and number of iterations
print(f'Final Weights: {weights}')
print(f'Final Bias: {bias}')
print(f'Number of Iterations Until Convergence: {iterations}')
```

Output




```
Final Weights: [0.1 0.1]
Final Bias: -0.1
Number of Iterations Until Convergence: 4
```